Change Point Detection in Time Series

Zhengxiao Wei, Farouk S. Nathoo, Eleah Stringer

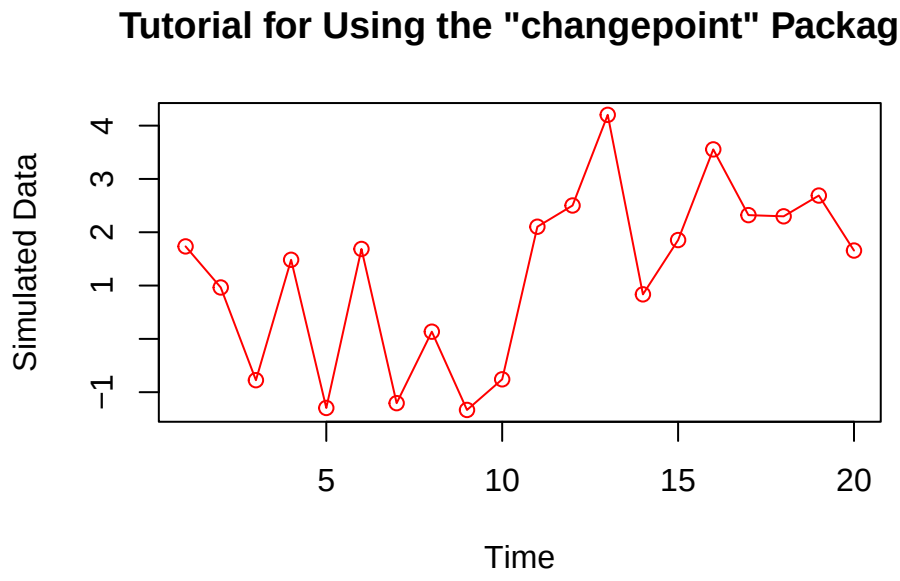
July 4 - September 29, 2023 (v2.2)

Change-point analysis is a statistical method used to detect points in ordered data where the characteristics have changed. This detection could suggest a deterministic change in the mean, variance, distribution, or some other parameter of interest.

1. Change in mean: Using the "*changepoint*" package (v 2.2.4)

Generate sample data where the population means for the first and second half data are 0 and 2, respectively. Perform the change-in-mean analysis using the *cpt.mean* function. The expected change-point should be the midpoint, i.e., 10.

```
set.seed(277); demo <- c(rnorm(10, 0, 1), rnorm(10, 2, 1))
plot(demo, type="o", col="red", xlab="Time", ylab="Simulated Data",
     main="Tutorial for Using the \"changepoint\" Package")
```




```
library(changepoint)
cpts(cpt.mean(demo, method="AMOC", penalty="MBIC"))
```

```
## [1] 10
```

- The `"AMOC"` method stands for "At Most One Change", meaning that the function will identify at most one change point by default.
- The `penalty` argument is to avoid overfitting. The modified Bayesian information criterion (`"MBIC"`) is used by default.

The real data are from an experiment to study the effect of intermittent fasting (IF) on the lymphocyte measurements of 15 participants, each of whom was repeatedly measured before, during, and after the intervention.

The goal is then to identify the change point in each participant's mean measurements and check whether it is after receiving the intervention.

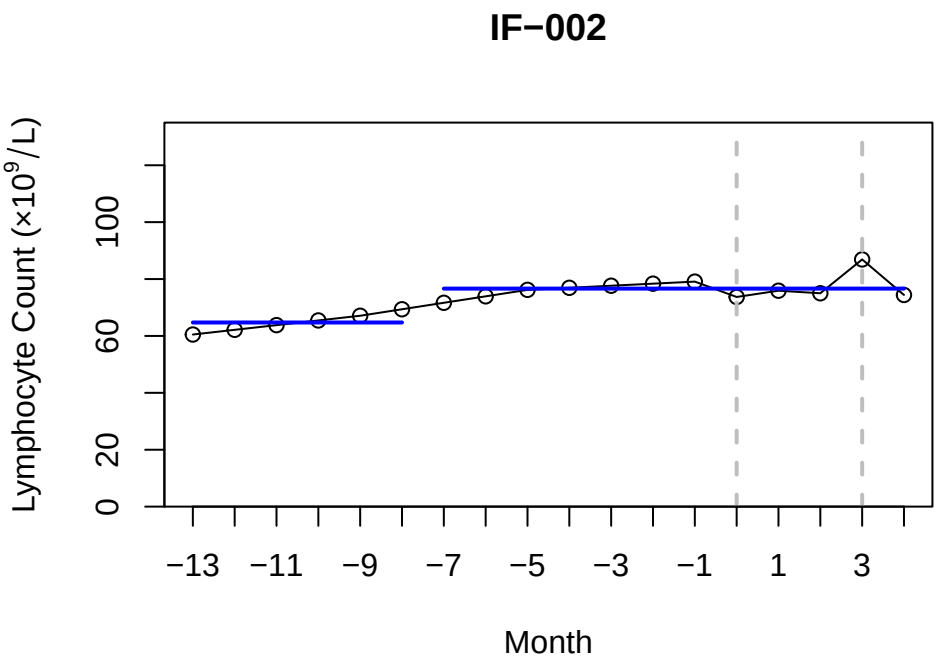
Key assumption: the package assumes a regularly spaced time series, but our pretest data (and some missing values during the intervention) do not satisfy this condition. Thus, we need to perform the **interpolation**, e.g., using the *approx* function.

```
df <- read.csv("~/Downloads/IF_v3.1.csv", stringsAsFactors=T)
dat <- df[!is.na(df$LYMP),] #IF-013 contains one missing value
id <- unique(dat$ID); phases <- vector("list", length(id))  #pre-allocation
ts_list <- replicate(length(id), list("x"=NULL, "y"=NULL, "raw"=NULL), simplify=F)
names(ts_list) <- names(phases) <- id
for (subj in id) {
  phases[[subj]] <- c(tail(subset(dat, ID==subj & Phase==0)[,"Time"],1), #start of fasting
                      tail(subset(dat, ID==subj & Phase==1)[,"Time"],1)) #start of posttest
  for (trt in unique(dat$Phase)) { #interpolate within each phase
    ts_raw <- subset(dat, ID==subj & Phase==trt)
    if (length(na.omit(ts_raw$LYMP)) > 1) {
      temp <- approx(x=ts_raw$Time, y=ts_raw$LYMP,
                     xout=c(min(ts_raw$Time):max(ts_raw$Time)))
      temp$raw <- temp$x %in% ts_raw$Time
    } else {temp <- list("x"=ts_raw$Time, "y"=ts_raw$LYMP, "raw"=T)}
    ts_list[[subj]] <- lapply(names(ts_list[[subj]]),
                              function(x) c(ts_list[[subj]][[x]], temp[[x]]))
    names(ts_list[[subj]]) <- c("x", "y", "raw")
  }
}
```

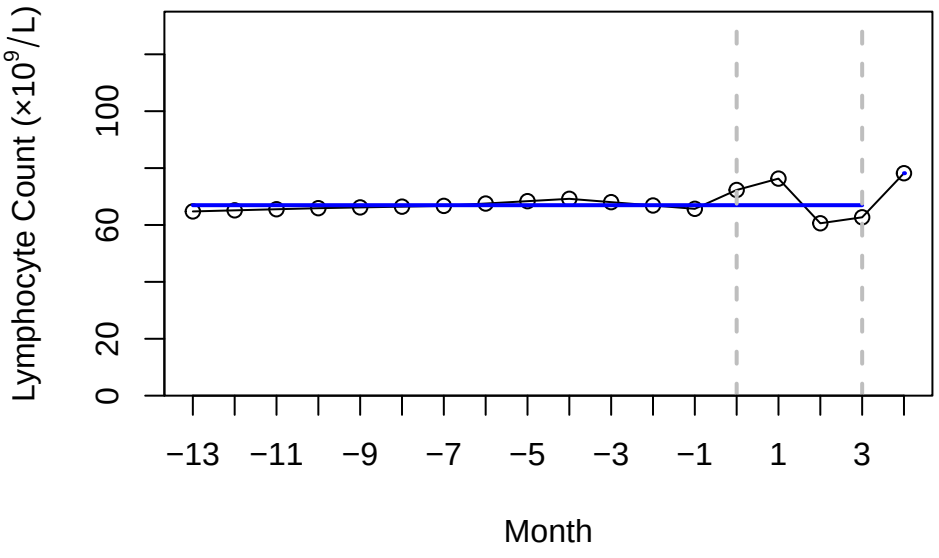
```
for (subj in id) {
  ts <- ts_list[[subj]]
  result <- cpt.mean(ts$y) #WRONG!
  index <- cpts(result) #index of the change point in the data

  par(mar=c(5,5,5,5))
  #blue lines indicate states; dashed gray lines indicate phases
  plot(result, type="o", cpt.col="blue", cpt.width=2, xaxt="n", ylim=c(5,130), main=subj,
       xlab="Month", ylab=expression(paste("Lymphocyte Count (\u00d7",10^9/L,")")))
  abline(v=which(ts$x %in% phases[[subj]]), col="gray", lty=2, lwd=2)
  axis(1, at=1:length(ts$y), labels=ts$x)

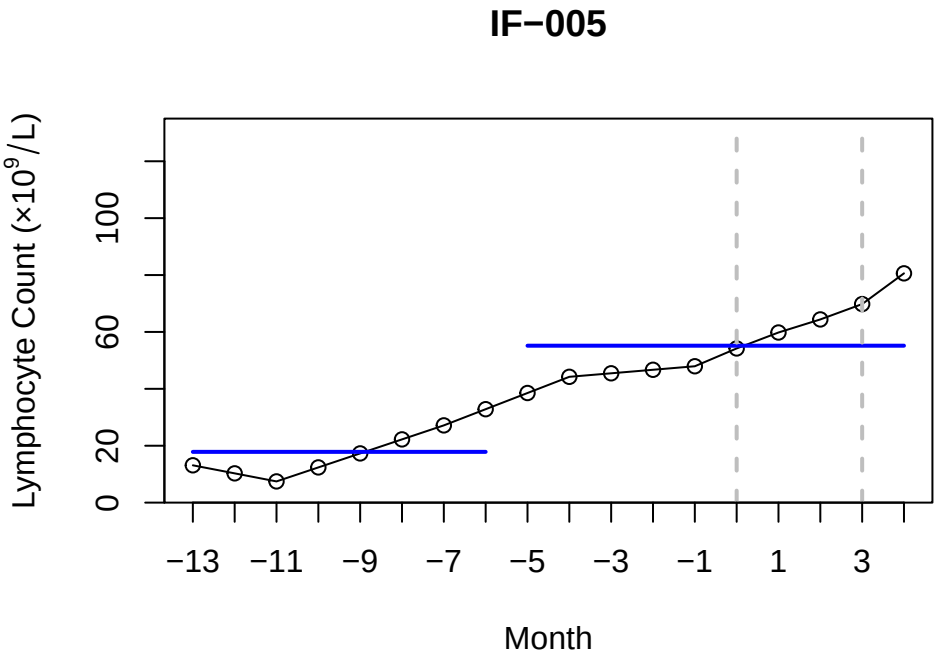
  cat(paste0("↑ The change point for ",subj," was Month ",ts$x[index],".\n"))
  cat(paste0("  The mean lymphocyte counts were ",
             round(param.est(result)$mean[1], 2)," and ",
             round(param.est(result)$mean[1], 2),".\n"))
  cat("  NOT FINALIZED YET!")
}
```



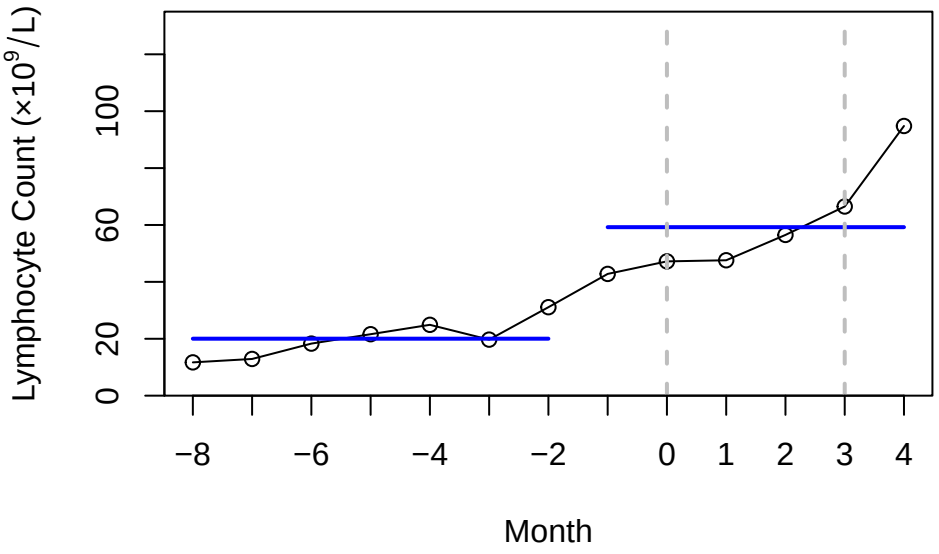

```
## ↑ The change point for IF-002 was Month -8.
##   The mean lymphocyte counts were 64.73 and 64.73.
##   NOT FINALIZED YET!
```



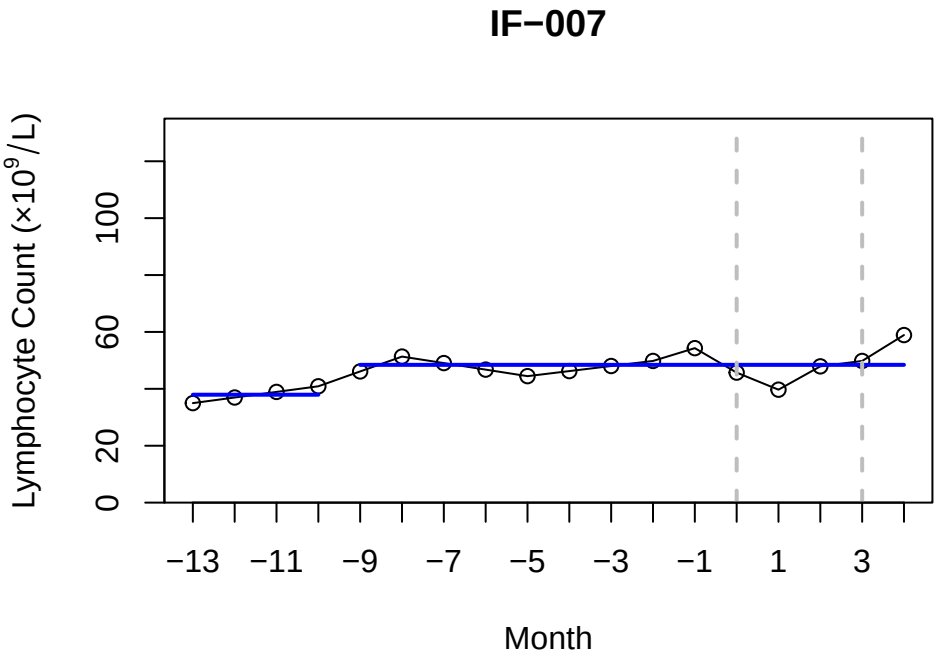

```
## ↑ The change point for IF-004 was Month 3.
##   The mean lymphocyte counts were 66.96 and 66.96.
##   NOT FINALIZED YET!
```



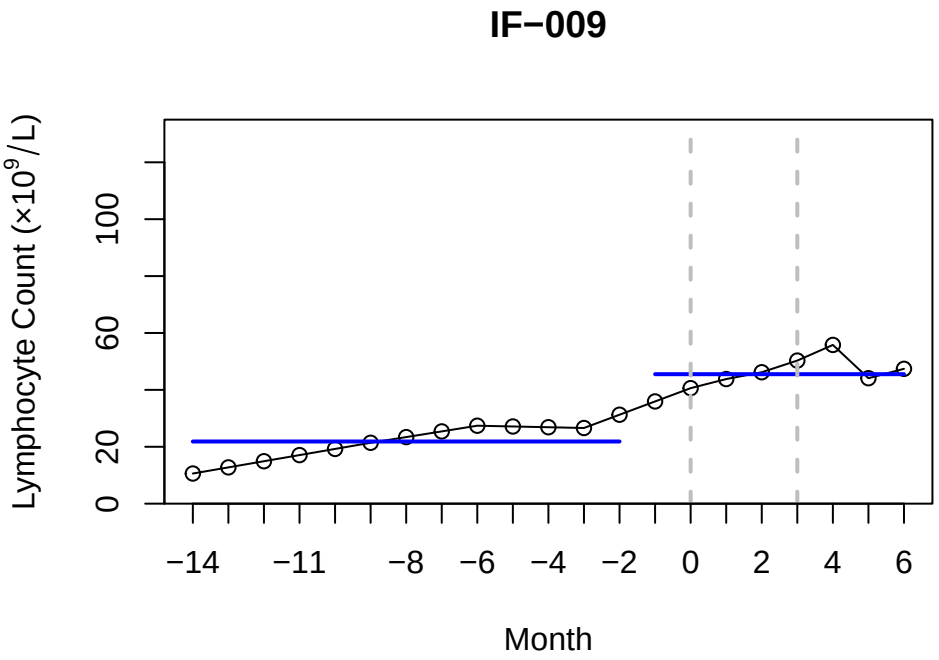

```
## ↑ The change point for IF-005 was Month -6.
##   The mean lymphocyte counts were 17.84 and 17.84.
##   NOT FINALIZED YET!
```



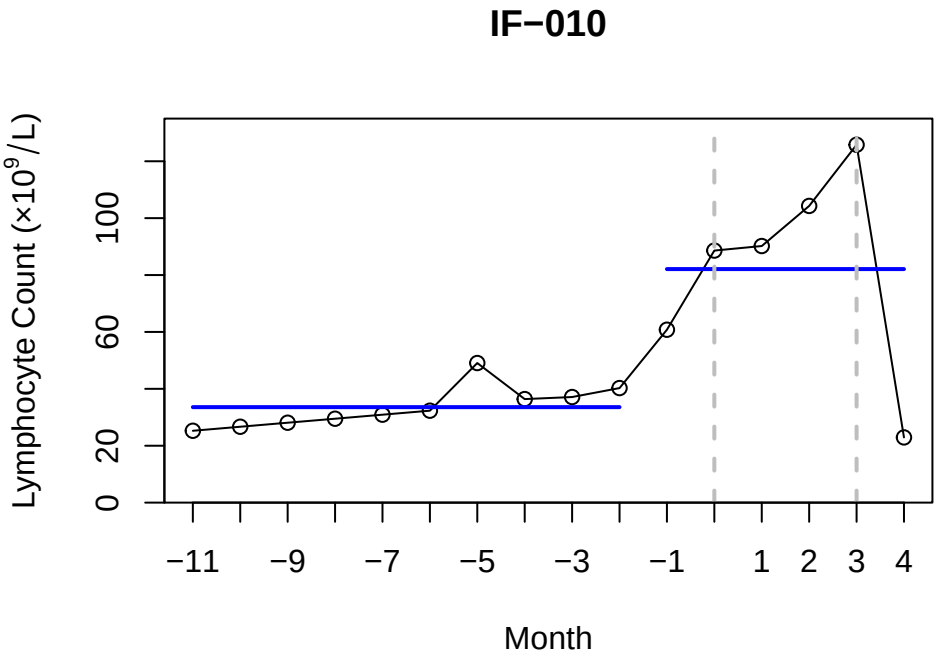

```
## ↑ The change point for IF-006 was Month -2.
##   The mean lymphocyte counts were 20.03 and 20.03.
##   NOT FINALIZED YET!
```



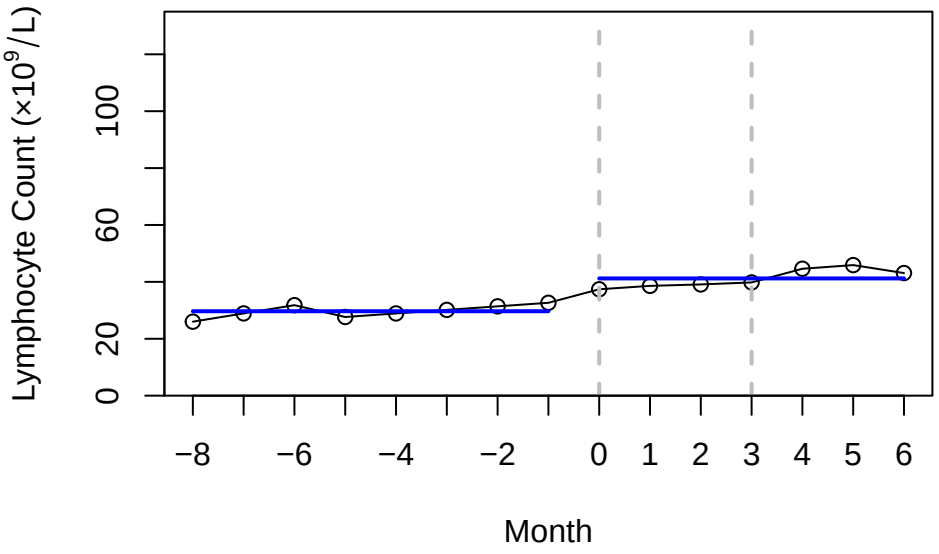

```
## ↑ The change point for IF-007 was Month -10.
##   The mean lymphocyte counts were 37.94 and 37.94.
##   NOT FINALIZED YET!
```



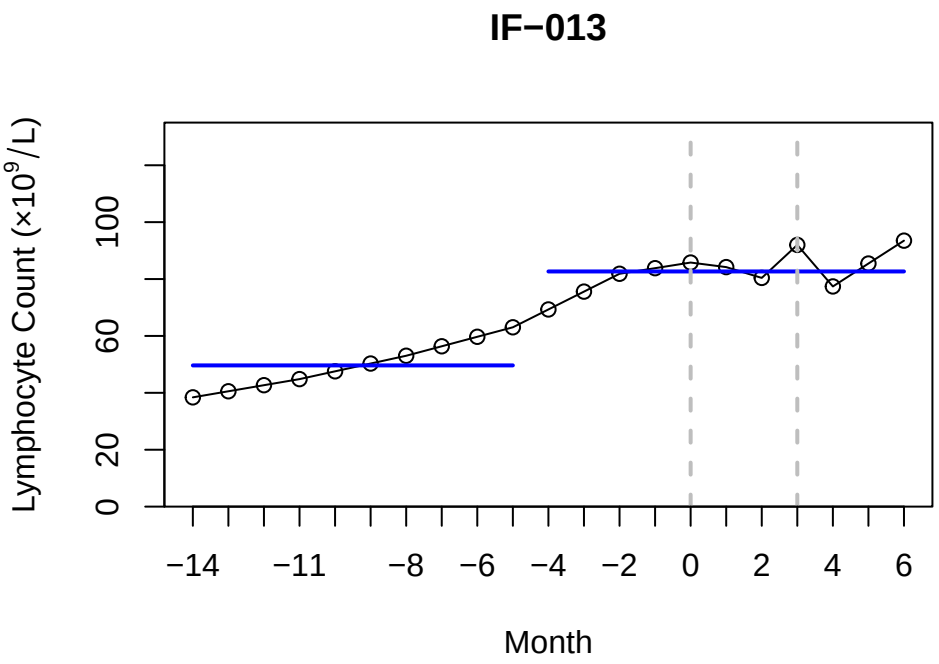

```
## ↑ The change point for IF-009 was Month -2.
##   The mean lymphocyte counts were 21.85 and 21.85.
##   NOT FINALIZED YET!
```



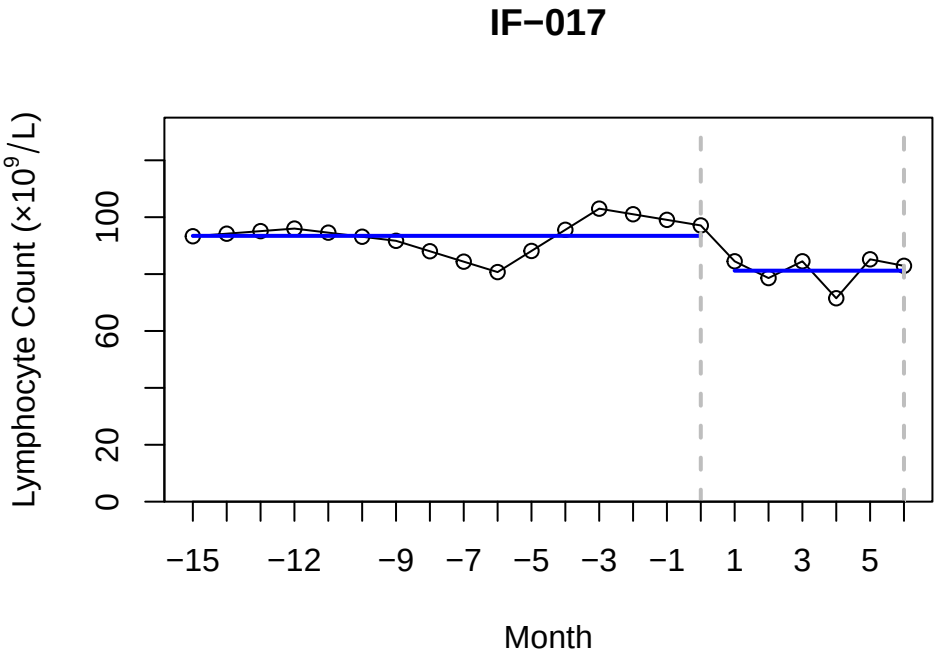

```
## ↑ The change point for IF-010 was Month -2.
##   The mean lymphocyte counts were 33.56 and 33.56.
##   NOT FINALIZED YET!
```




```
## ↑ The change point for IF-012 was Month -1.
##   The mean lymphocyte counts were 29.68 and 29.68.
##   NOT FINALIZED YET!
```

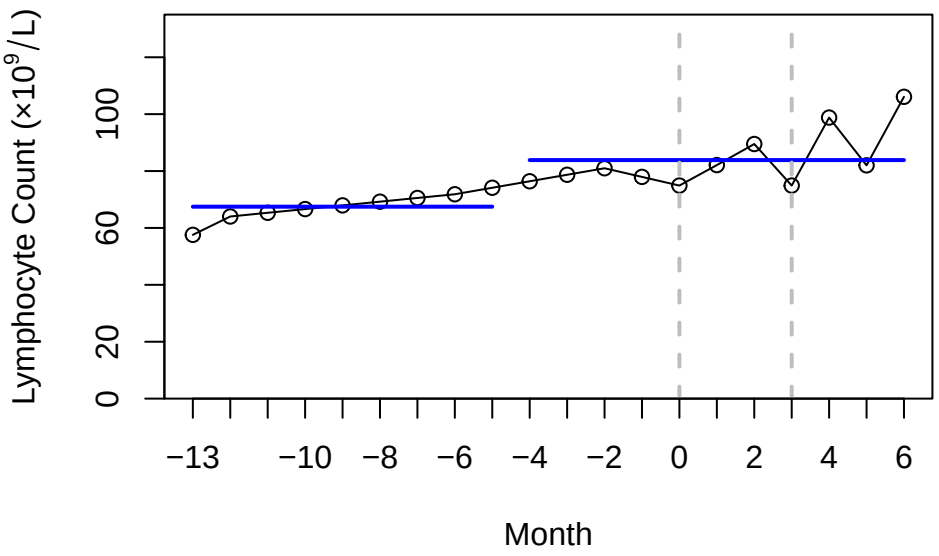



```
## ↑ The change point for IF-013 was Month -5.
##   The mean lymphocyte counts were 49.64 and 49.64.
##   NOT FINALIZED YET!
```



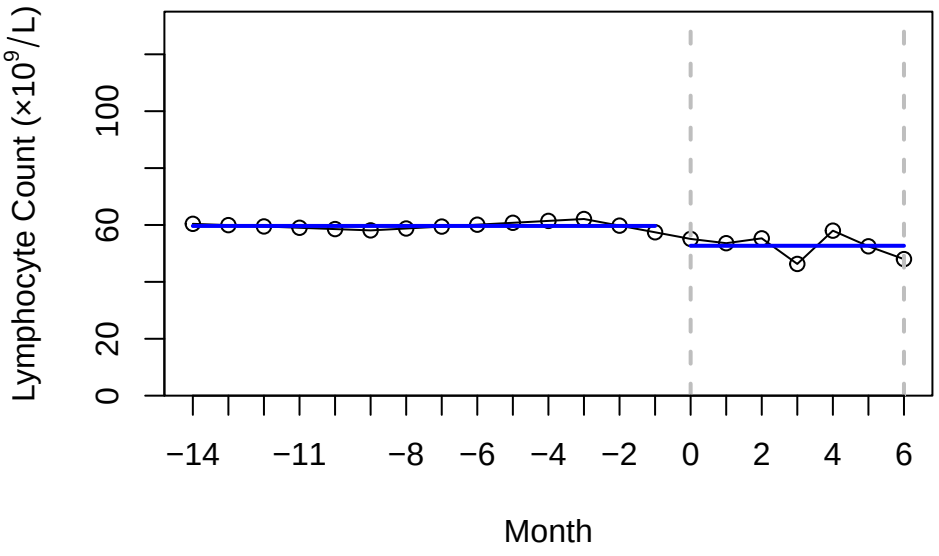

```
## ↑ The change point for IF-017 was Month 0.
##   The mean lymphocyte counts were 93.44 and 93.44.
##   NOT FINALIZED YET!
```

IF−018

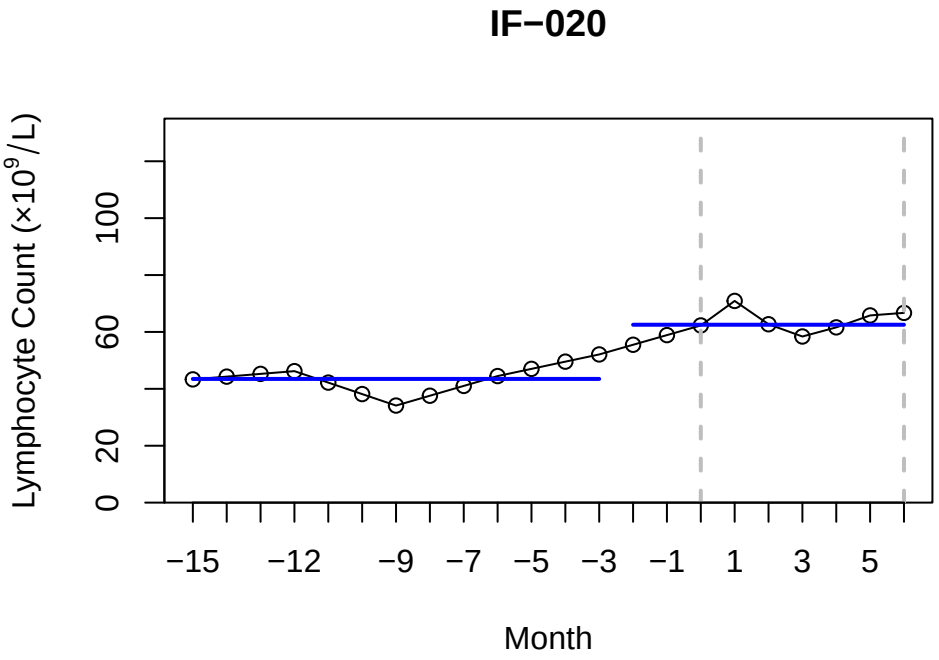



```
## ↑ The change point for IF-018 was Month -5.
##   The mean lymphocyte counts were 67.47 and 67.47.
##   NOT FINALIZED YET!
```

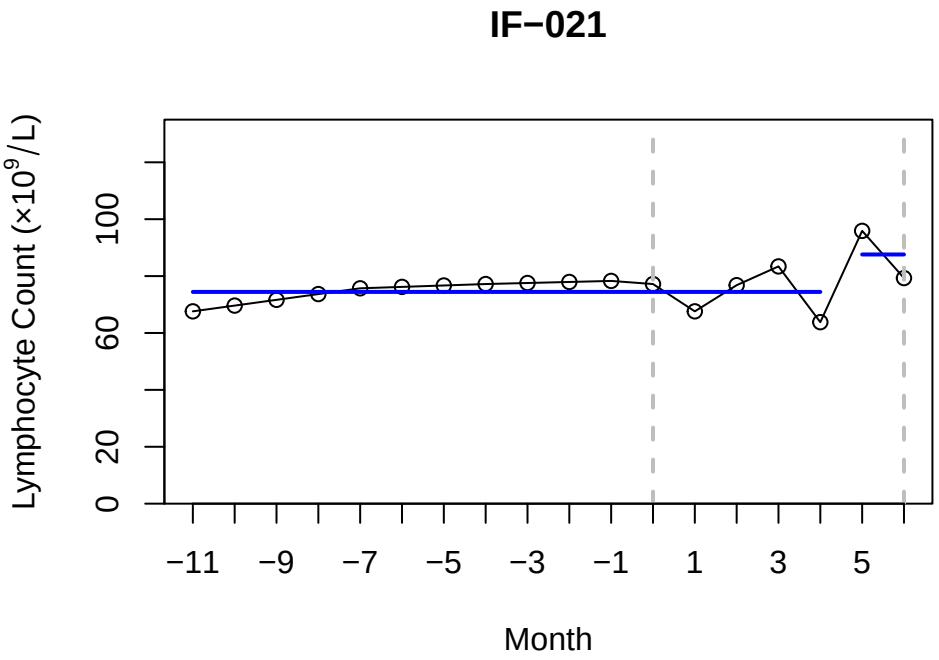
IF−019



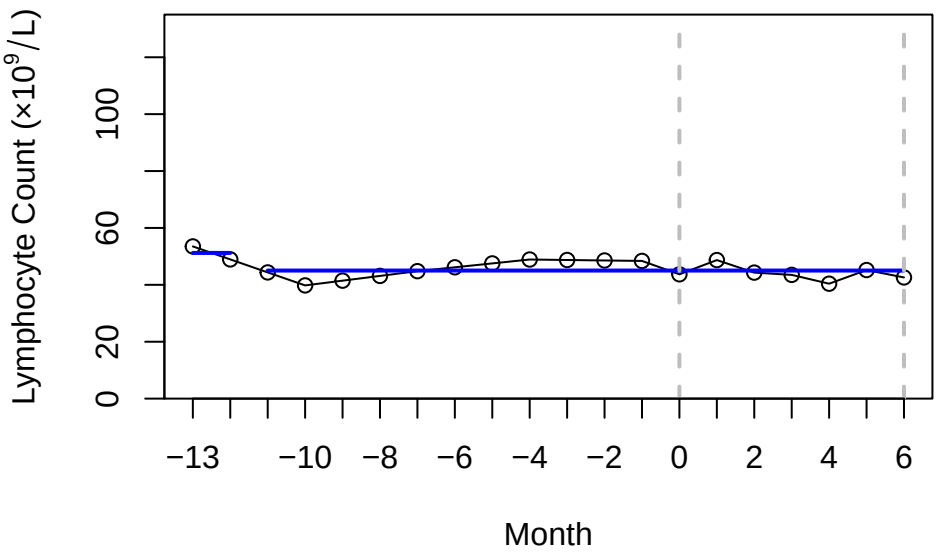

```
## ↑ The change point for IF-019 was Month -1.
##   The mean lymphocyte counts were 59.66 and 59.66.
##   NOT FINALIZED YET!
```




```
## ↑ The change point for IF-020 was Month -3.
##   The mean lymphocyte counts were 43.48 and 43.48.
##   NOT FINALIZED YET!
```




```
## ↑ The change point for IF-021 was Month 4.
##   The mean lymphocyte counts were 74.43 and 74.43.
##   NOT FINALIZED YET!
```




```
## ↑ The change point for IF-022 was Month -12.
##   The mean lymphocyte counts were 51.22 and 51.22.
##   NOT FINALIZED YET!
```

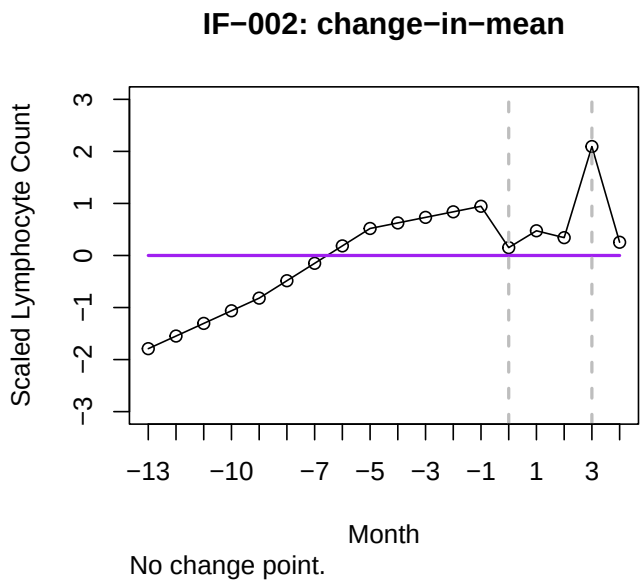
Do not jump to conclusions.

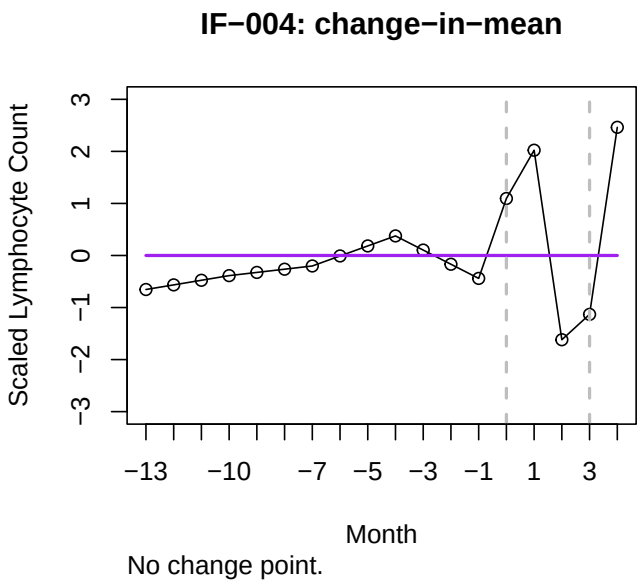
Do not jump to conclusions.

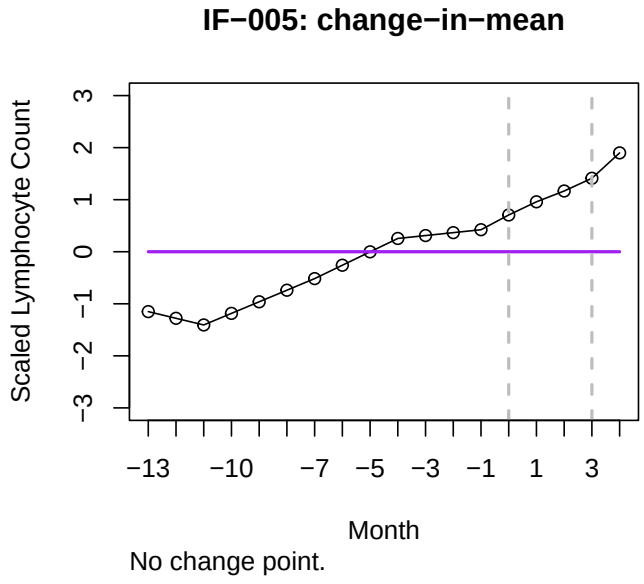
Do not jump to conclusions.

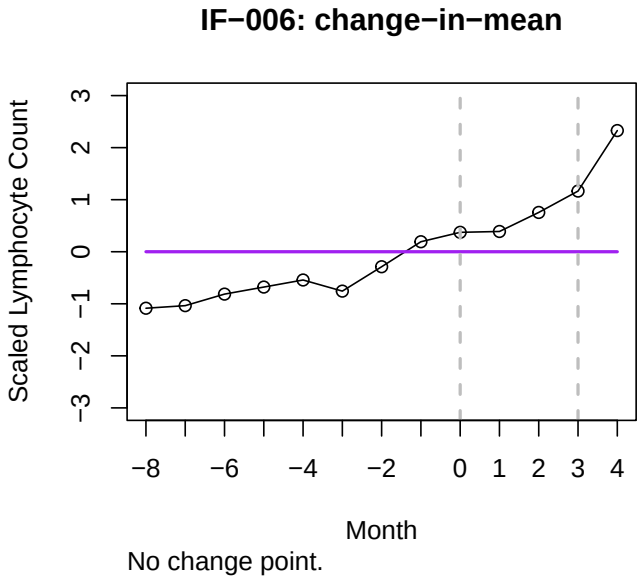
Importantly, the function assumes that the variance of a time series is 1 (Killick, 2020). We may need to **scale** the data.

```
par(mfrow=c(1,2))
for (subj in id) {
  ts <- ts_list[[subj]]
  result <- cpt.mean(c(scale(ts$y)))
  index <- cpts(result)
  if(length(index) > 0) {
    tex <- paste0("The change point was Month ",ts$x[index],".")
  } else {tex <- paste0("No change point.")}
  plot(result, type="o", cpt.col="purple", cpt.width=2, xaxt="n", ylim=c(-3,3),
       xlab="Month", ylab="Scaled Lymphocyte Count", main=paste0(subj, ": change-in-mean"))
  mtext(tex, side=1, adj=0, line=4)
  abline(v=which(ts$x %in% phases[[subj]]), col="gray", lty=2, lwd=2)
  axis(1, at=1:length(ts$y), labels=ts$x)
}
```

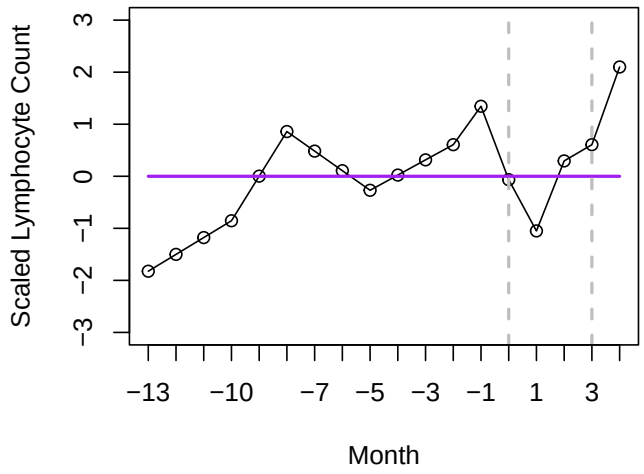






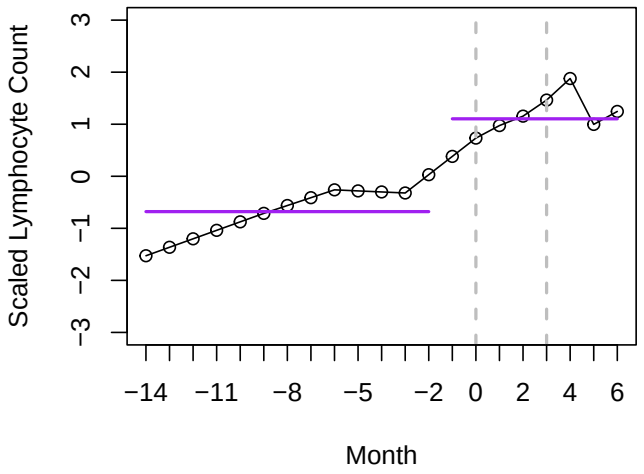


IF−007: change−in−mean



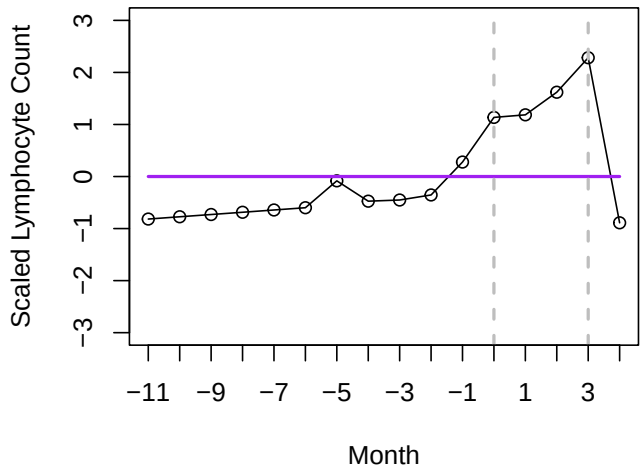
No change point.

IF−009: change−in−mean



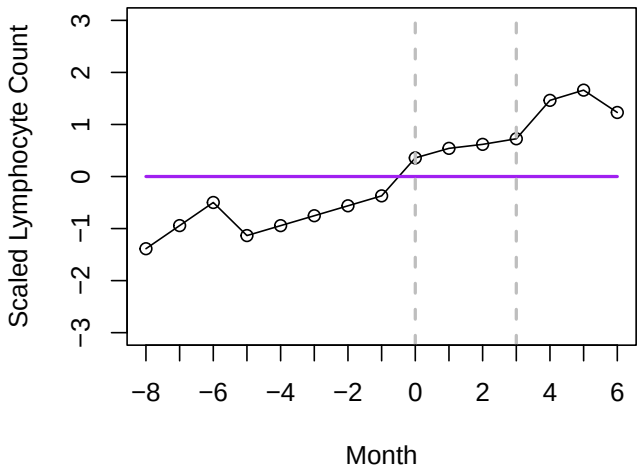
The change point was Month −2.

IF−010: change−in−mean



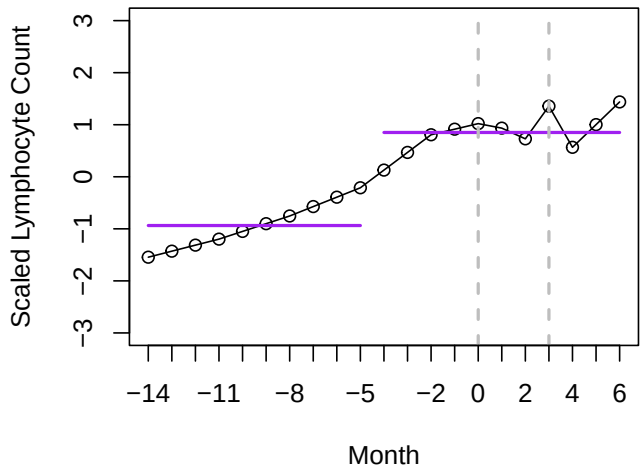
No change point.

IF−012: change−in−mean



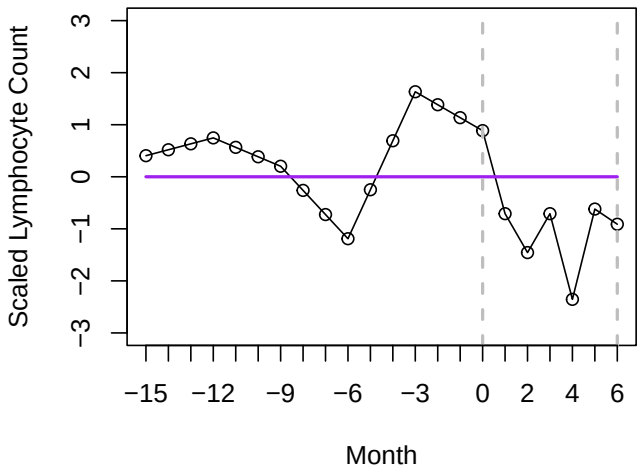
No change point.

IF−013: change−in−mean



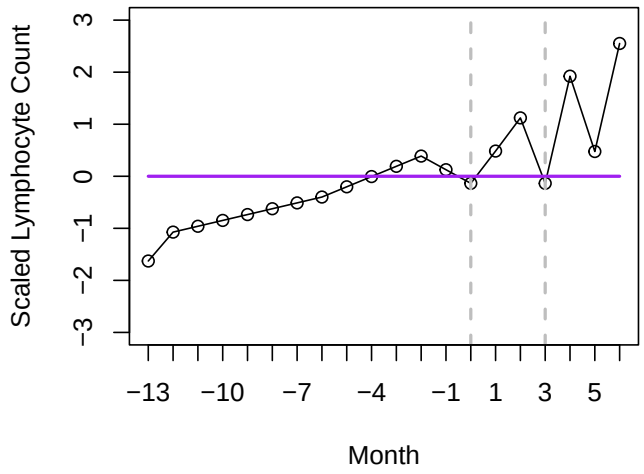
The change point was Month −5.

IF−017: change−in−mean



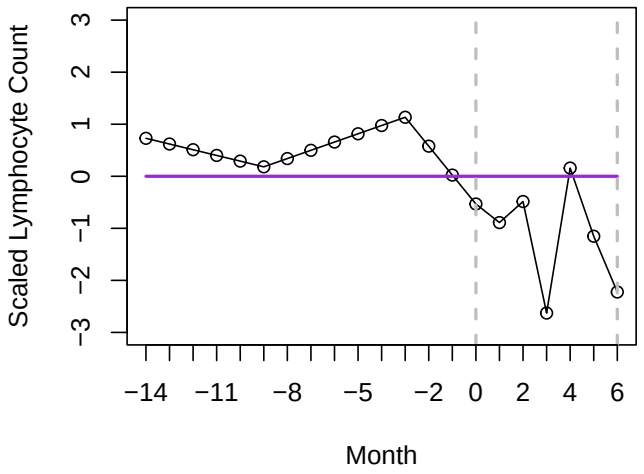
No change point.

IF−018: change−in−mean



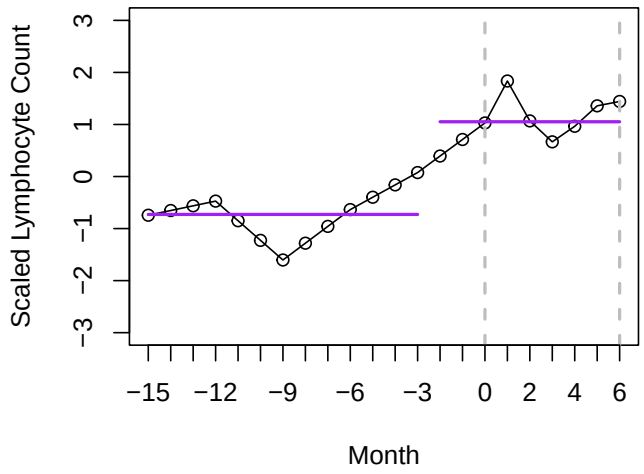
No change point.

IF−019: change−in−mean



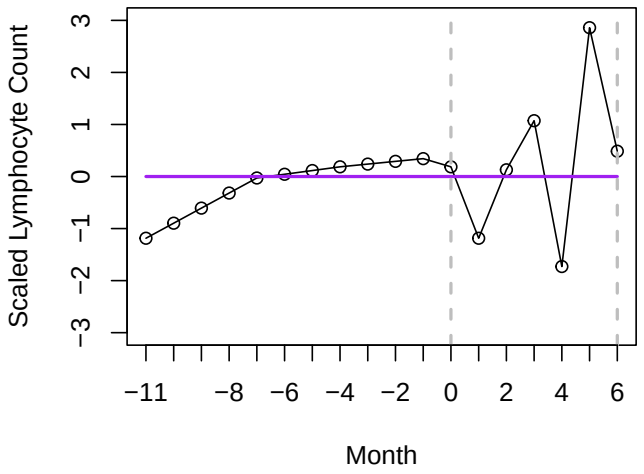
No change point.

IF−020: change−in−mean



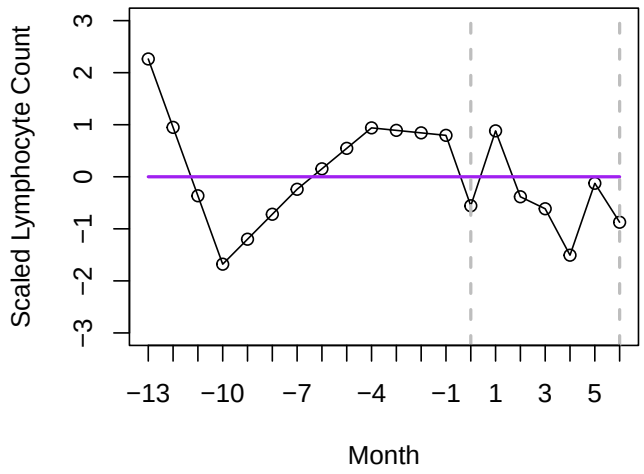
The change point was Month −3.

IF−021: change−in−mean



No change point.

IF−022: change−in−mean



No change point.

If we want to find two or more change points, we should use the `"PELT"` method (Pruned Exact Linear Time). $\mathcal{O}(n^2)$ at worst.

```
for (subj in id) {
  ts <- ts_list[[subj]]
  index <- cpts(cpt.mean(c(scale(ts$y)), method="PELT"))
  if (length(index) == 0) {
    cat(paste0("No change point for ",subj,".\n"))
  } else if (length(index) == 1) {
    cat(paste0("The change point for ",subj," was Month ",
               ts$x[index],".\n"))
  } else {
    cat(paste0("The change points for ",subj," were Months ",
               paste(ts$x[index], collapse=", "),".\n"))
  }
}
```

```
## The change point for IF-002 was Month -8.
## No change point for IF-004.
## The change point for IF-005 was Month -6.
## No change point for IF-006.
## No change point for IF-007.
## The change point for IF-009 was Month -2.
## No change point for IF-010.
## The change point for IF-012 was Month -1.
## The change point for IF-013 was Month -5.
## No change point for IF-017.
## No change point for IF-018.
## The change point for IF-019 was Month -1.
## The change point for IF-020 was Month -3.
## No change point for IF-021.
## No change point for IF-022.
```

Other methods:

- `"SegNeigh"`: Segment Neighborhood.

A variation of the exhaustive search method looks at a subset of points around the maximum likelihood estimate of the change point rather than every possible point. It can be computationally expensive, $\mathcal{O}(Qn^2)$ where Q specifies the maximum number of change points.

- `"BinSeg"`: Binary Segmentation.

It iteratively splits the data into two segments at the point that maximizes some criterion (e.g., the likelihood), then continues to split the resulting segments similarly until we detect no more significant changes. It tends to find the most significant changes first and can miss less significant ones nearby. $\mathcal{O}(n \log n)$ in CPU time.

2. Change in regression: Using the "*strucchange*" package (v 1.5-3)

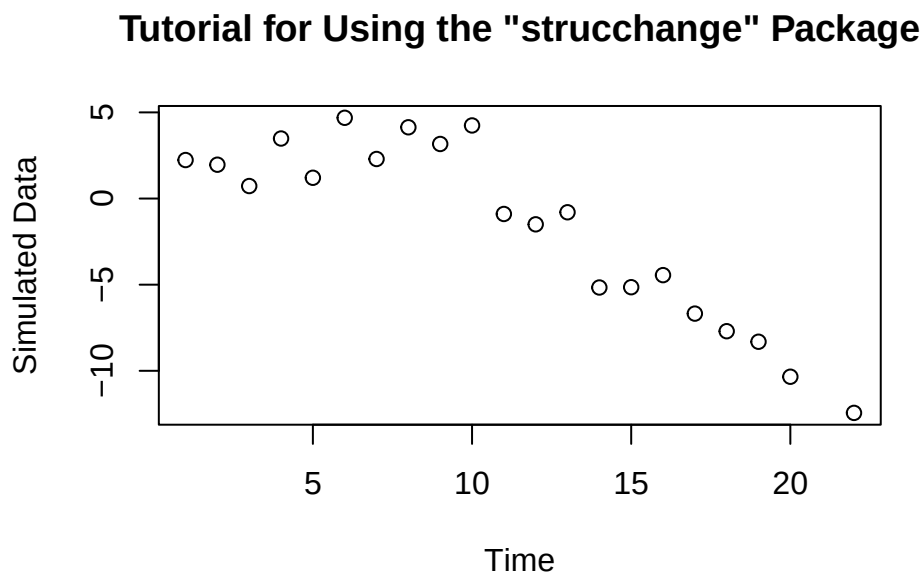
The *breakpoints* function implements the Bai-Perron method of multiple structural breakpoints. This global optimization procedure considers all possible data partitions into segments and finds the partition that minimizes the sum of squared residuals (RSS) or BIC within each segment.

In addition to changes in mean, it seeks to identify points in time at which the coefficients of the linear regression model change.

```
library(strucchange)
breakpoints(demo ~ 1)$breakpoints #consistent with previous p. 1
```

```
## [1] 10
```

```
x <- c(1:20,22) #irregularly spaced (or missing value)
set.seed(277); y <- c(0.5*x[1:10], 10-x[11:21]) + rnorm(21)
fit <- breakpoints(y ~ x) #simple linear regression
plot(y ~ x, xlab="Time", ylab="Simulated Data",
     main="Tutorial for Using the \"strucchange\" Package")
```




```
fit$breakpoints
```

```
## [1] 10
```

```
subj <- "IF-013" #IF-009, 013, 017, (018), 019, 020, 022
ts_raw <- subset(dat, ID==subj)
table(dat$ID)
```

```
##
## IF-002 IF-004 IF-005 IF-006 IF-007 IF-009 IF-010 IF-012 IF-013 IF-017 IF-018
##      9     10     10     12     11     11     12     11     11     12     11
## IF-019 IF-020 IF-021 IF-022
##     10     12     11     12
```

```
breakpoints(LYMP ~ Time, data=ts_raw)
```

```
## Error in breakpoints.formula(LYMP ~ Time, data = ts_raw): minimum segment size must be greater than the number of regressors
```

Though breakpoint detection can tackle irregularly spaced time series, our data must be longer to be split into segments. Error: minimum segment size must be greater than the number of regressors.

```
ts <- ts_list[[subj]]
n <- length(ts$y)
(fit <- breakpoints(y ~ x, data=ts))
```

```
##
##   Optimal 4-segment partition:
##
## Call:
## breakpoints.formula(formula = y ~ x, data = ts)
##
## Breakpoints at observation number:
## 10 15 18
##
## Corresponding to breakdates:
## 0.4761905 0.7142857 0.8571429
```

```
#crosses indicate breakpoints at observation number
bp.pch <- ifelse(1:n %in% fit$breakpoints, 4, 1)
tex2 <- paste0("The change points were Months ",
               paste(ts$x[fit$breakpoints], collapse=", "),".\n")
par(mar=c(5,5,5,5))
plot(c(1:n)/n, ts$y, xlab="Month", col=ifelse(ts$raw, "black", "gray"),
     xaxt="n", pch=bp.pch, ylim=c(5,130), main=paste0(subj, ": change-in-regression"),
     ylab=expression(paste("Lymphocyte Count (\u00d7",10^9/L,")")))
mtext(tex2, side=1, adj=0, line=5); lines(fitted(fit), col="purple")
abline(v=which(ts$x %in% phases[[subj]])/n, col="gray", lty=2, lwd=2)
axis(1, at=c(1:n)/n, labels=ts$x); text(x=16.5/n, y=125, labels="IF", col="red")
```

